

UNIVERSIDAD DEL QUINDÍO

FACULTAD DE INGENIERÍA

Ingeniería de Sistemas y Computación

PROYECTO INTEGRADOR - TurismoUQ

Plataforma de reservas turísticas del Quindío

ENTREGA 1 - Modelo y consultas de análisis

Modelo · DDL · Carga de datos · Tablespace · 8 consultas · Vista materializada

Campo	Valor
Asignatura	Bases de Datos II · código 12338
Periodo	2026-2
Grupo	01D
Docente	Ana Maria Vanegas Rojas
Integrante 1	Juan Felipe Hurtado Londoño
Integrante 2	Santiago Marin Serna
Integrante 3	Sebastián Agudelo Mendez
Motor	Oracle Database 21c Express Edition (XE)
Repositorio	https://github.com/FelipeHurtadoo/turismouq-bd2

1. Introducción y alcance

TurismoUQ es la base de datos de una plataforma que reúne la oferta de alojamiento de los 12 municipios del departamento del Quindío (fincas cafeteras, hoteles, glampings, hostales y otros) y gestiona reservas, pagos y servicios complementarios.

El entregable de este proyecto no es una interfaz de usuario, sino la base de datos: el modelo, la lógica que vive dentro del motor y su administración. La Entrega 1 comprende el modelo de datos, el script DDL completo, la carga del volumen mínimo exigido, un tablespace propio para el histórico de reservas, las ocho consultas de análisis y la vista materializada.

Toda la solución fue diseñada desde el principio para soportar las Entregas 2 y 3 sin reconstruir el modelo. Dos decisiones estructurales sostienen ese objetivo y se desarrollan en la sección 4: cómo una reserva toma varias habitaciones, y cómo una estadía atraviesa varias temporadas con tarifas distintas.

1.1 Entorno de ejecución

Oracle Database 21c Express Edition sobre contenedor Docker, base XE, PDB XEPDB1. Se optó por esta modalidad porque el instalador nativo de Oracle XE 21c para Windows no soporta las ediciones Home de Windows, según la documentación oficial de Oracle. El motor, la versión y las características son las mismas que en una instalación nativa.

1.2 Convención del documento

[R] Requisito literal del documento oficial del proyecto.

[D] Decisión de diseño del equipo. No proviene del enunciado; se justifica en cada caso.

2. Análisis de requisitos

La siguiente tabla relaciona cada requisito del documento oficial con el artefacto que lo resuelve.

#	Requisito	Entrega	Dónde se resuelve
R1	14 entidades mínimas; se puede agregar, no quitar	1	04_ddl.sql
R2	DDL con PK, FK, CHECK, NOT NULL y UNIQUE	1	04_ddl.sql
R3	Volumen: 12 / 60 / 400 / 3.000 / 25.000 / 40.000	1	05 y 06
R4	Reservas repartidas en 2024–2026	1	06, bloque C
R5	Datos generados con PL/SQL y DBMS_RANDOM	1	05 y 06
R6	Tablespace propio para el histórico + justificación	1	01, sección 6
R7	PIVOT: ocupación por municipio y mes	1	08, consulta 1
R8	ROLLUP/CUBE con GROUPING: ingresos	1	08, consulta 2
R9	RANK con PARTITION BY: top 3 por municipio	1	08, consulta 3
R10	LAG: variación mes contra mes	1	08, consulta 4
R11	Consulta parametrizada con variables de enlace	1	08, consulta 5
R12	Vista materializada + política de refresco	1	07
R13	UNPIVOT	1	08, consulta 7

#	Requisito	Entrega	Dónde se resuelve
R14	Consulta libre de negocio	1	08, consulta 8
R25	Scripts numerados que corran en base limpia	1	Sección 10
R26	PDF con el modelo y las decisiones de diseño	1	Este documento

Las siguientes ambigüedades del enunciado se resolvieron con decisiones de diseño explícitas:

Ambigüedad	Decisión adoptada [D]
El documento no define "ocupación"	Noches-habitación vendidas ÷ noches-habitación disponibles, que es el indicador estándar de la industria hotelera y el único que permite comparar municipios con oferta muy distinta
El documento no define "ingreso"	Ingreso devengado: suma de las tarifas noche a noche de reservas CONFIRMADA o COMPLETADA. No se usan los pagos, porque un pago puede ir por anticipos, cruzarse de mes y no sabe a qué temporada pertenece la noche que financia
No indica si la reserva tiene fechas o las tiene cada habitación	Las fechas viven en la cabecera RESERVA: una reserva es una estadía
No indica el grano de la tarifa	(alojamiento, categoría, temporada). Ver sección 4.3
No indica si las temporadas cubren todo el calendario	Sí: disjuntas y sin huecos. Es condición necesaria para que ninguna noche quede sin tarifa

3. Modelo conceptual y lógico

3.1 Arquitectura en tres capas

El modelo se organiza en tres capas con comportamientos distintos, y esa separación es la que justifica la distribución física en tablespaces de la sección 6.

Capa	Tablas	Comportamiento	Tablespace
Catálogo	MUNICIPIO, TIPO_ALOJAMIENTO, CATEGORIA_HABITACION, ALOJAMIENTO, HABITACION, CLIENTE, SERVICIO, USUARIO_SISTEMA	Poco volumen, cambia poco, se lee por clave	TS_TURISMO_DAT
Temporal	TEMPORADA, DIM_TIEMPO, TARIFA	Calendario y precios; se carga una vez por periodo	TS_TURISMO_DAT
Histórico	RESERVA, RESERVA_HABITACION, PAGO, RESERVA_SERVICIO, RESENA, AUDITORIA_TARIFA	Solo crece, inmutable al cerrarse, se lee en barridos analíticos	TS_TURISMO_HIST

3.2 Entidades del modelo

El documento exige 14 entidades mínimas, todas presentes. Se agregaron tres adicionales, permitidas explícitamente por el enunciado y justificadas en la sección 4.

Origen	Entidades
Exigidas [R]	MUNICIPIO, TIPO_ALOJAMIENTO, ALOJAMIENTO, HABITACION, TEMPORADA, TARIFA, CLIENTE, RESERVA, RESERVA_HABITACION, PAGO, SERVICIO, RESERVA_SERVICIO, RESENA, USUARIO_SISTEMA
Agregadas [D]	CATEGORIA_HABITACION (convierte un CHECK de texto en una FK real), DIM_TIEMPO (calendario 2024–2026 con su temporada), AUDITORIA_TARIFA (destino del disparador de la Entrega 2; vacía en esta entrega)

3.3 Relaciones y cardinalidades

Relación	Cardinalidad	Observación
MUNICIPIO – ALOJAMIENTO	1:N	Obligatoria
TIPO_ALOJAMIENTO – ALOJAMIENTO	1:N	Obligatoria
ALOJAMIENTO – HABITACION	1:N	Obligatoria
CATEGORIA_HABITACION – HABITACION	1:N	Determina la tarifa aplicable
ALOJAMIENTO – TARIFA	1:N	Obligatoria
TEMPORADA – TARIFA	1:N	Obligatoria
TEMPORADA – DIM_TIEMPO	1:N	Cada día tiene exactamente una temporada
CLIENTE – RESERVA	1:N	Obligatoria
ALOJAMIENTO – RESERVA	1:N	Obligatoria
RESERVA – HABITACION	N:M	Resuelta por RESERVA_HABITACION
RESERVA – PAGO	1:N	Opcional: una reserva pendiente aún no pagó
RESERVA – SERVICIO	N:M	Resuelta por RESERVA_SERVICIO
RESERVA – RESENA	1:0..1	UNIQUE sobre la FK impone el máximo de una
ALOJAMIENTO – USUARIO_SISTEMA	1:N	Opcional: gerente y auditor no tienen alojamiento

La caja verde MV_OCUPACION_MENSUAL corresponde a la vista materializada y aparece sin conexiones porque no participa en relaciones de integridad referencial.

4. Decisiones de diseño

4.1 Decisión A: una reserva con varias habitaciones

Esta es una de las dos decisiones que el documento exige defender.

Alternativas evaluadas

Alternativa	Ventaja	Por qué se descarta
FK directa RESERVA.id_habitacion	Implementación trivial	Solo admite una habitación. Una familia que toma tres cabañas necesitaría tres reservas, tres pagos y tres reseñas para una misma estadía, repitiendo toda la cabecera
Lista de habitaciones en un VARCHAR2 separada por comas	Cabe en una sola columna	Viola la primera forma normal. Imposible de unir, de indexar y de validar con una clave foránea
Tabla puente RESERVA_HABITACION	Correcta y extensible	ELEGIDA

Diseño elegido

```
RESERVA (1) -----< RESERVA_HABITACION >----- (1) HABITACION
```

RESERVA_HABITACION no es una tabla puente vacía: tiene atributos propios del vínculo, huespedes (cuántas personas van en esa habitación) y valor_noches (cuánto costó esa habitación en esa estadía). Eso confirma que la relación N:M entre reservas y habitaciones es una entidad asociativa legítima y no un artificio de implementación.

Restricciones que la sostienen:

- uq_rh_reserva_hab UNIQUE (id_reserva, id_habitacion): la misma habitación no puede aparecer dos veces en la misma reserva.
- fk_rh_reserva ... ON DELETE CASCADE: las líneas no sobreviven a su cabecera.
- La validación "la habitación pertenece al alojamiento de la reserva" no puede ser una clave foránea, porque sería una dependencia transitiva. Queda para sp_crear_reserva en la Entrega 2. El script de validación comprueba que los datos cargados la cumplen: 0 violaciones.

En los datos cargados, 25.000 reservas generaron 28.539 líneas de habitación. Esa diferencia de 3.539 son precisamente las reservas que tomaron más de una habitación, y es la evidencia de que el modelo lo soporta.

4.2 Decisión B: una estadía que atraviesa varias temporadas

Es la segunda decisión que el documento exige defender, y el núcleo técnico del proyecto.

La tabla RESERVA no tiene columna id_temporada.

Incluirla sería el error clásico: obligaría a elegir una sola temporada para una estadía del 13 al 18 de diciembre, que es mitad temporada baja y mitad alta. Cualquier criterio de elección (la temporada del check-in, la mayoritaria) falsearía el ingreso reportado.

Diseño elegido: la temporada es una propiedad del calendario

```

TEMPORADA (rango de fechas, disjunto y sin huecos)
| 1:N
DIM_TIEMPO (un día = una fila = una temporada)
| unión por rango:
| d.fecha >= r.fecha_checkin AND d.fecha < r.fecha_checkout
RESERVA

```

El valor de una estadía se calcula así: por cada noche del rango, el día determina su temporada mediante DIM_TIEMPO, y la terna (alojamiento, categoría, temporada) determina la tarifa de esa noche concreta. La suma de esas tarifas es el valor correcto, sin importar cuántas temporadas atraviere la estadía.

Consecuencias que sostienen la decisión

- El rango es semiabierto [checkin, checkout). La noche de salida no se cobra: del 10 al 13 son tres noches, no cuatro. Por eso la unión usa >= checkin y < checkout, nunca BETWEEN.
- Las temporadas cubren el calendario sin huecos ni superposiciones. Si faltara un día, esa noche no tendría tarifa y la unión la descartaría en silencio: el total saldría más barato sin generar ningún error. Por eso el generador construye las temporadas comprimiendo una clasificación día a día, y el script de validación verifica ambas propiedades.
- La función fn_valor_estadia de la Entrega 2 no exige ningún cambio de modelo: es exactamente esta lógica, ya implementada en SQL en el bloque D del script de carga, encapsulada en PL/SQL.

Evidencia en los datos cargados

El script de validación confirma que el escenario no es teórico:

Temporadas atravesadas	Líneas de reserva	Valor promedio (COP)
1	23.602	894.883
2	4.883	1.130.642
3	54	1.540.407

Es decir, 4.937 estadías cruzan dos o más temporadas y fueron valoradas noche por noche con la tarifa correspondiente a cada una.

4.3 Grano de la tarifa [D]

TARIFA se define sobre la terna (id_alojamiento, categoria, id_temporada), con una restricción UNIQUE sobre esas tres columnas.

Alternativa	Filas aprox.	Evaluación
Tarifa por habitación × temporada	~40.000	Redundante: dos habitaciones estándar del mismo hotel siempre valen lo mismo. Actualizar precios exigiría tocar N filas por hotel, con riesgo de desincronizarlas
Tarifa por tipo de alojamiento × temporada	~600	Insuficiente: dos fincas cafeteras del mismo tipo no cobran lo mismo
(alojamiento, categoría, temporada)	12.960	ELEGIDA. Sin redundancia y con precio determinista para cualquier habitación

Como `fn_valor_estadia` recibe un `id_habitacion`, la resolución es `HABITACION → id_alojamiento + categoria → TARIFA`. El índice único `uq_tarifa_natural` es precisamente el camino de acceso de esa función en la Entrega 2.

4.4 Entidades adicionales [D]

CATEGORIA_HABITACION

La categoría es a la vez atributo de `HABITACION` y parte de la clave de `TARIFA`. Si fuera un `VARCHAR2` con un `CHECK` en cada tabla, nada garantizaría que ambas listas coincidan. Como tabla con clave foránea, la integridad la impone el motor.

DIM_TIEMPO

Es la única entidad adicional que cambia la forma de consultar, por lo que merece justificación explícita. La alternativa era generar las fechas al vuelo con `CONNECT BY`. Se descartó porque obligaría a repetir esa generación en cada consulta y en la función de la Entrega 2, impediría indexar y calcular estadísticas sobre las fechas, y sobre todo no permitiría almacenar a qué temporada pertenece cada día, forzando una unión por rango contra `TEMPORADA` en cada consulta en vez de una igualdad. Es la tabla de dimensión de tiempo clásica de cualquier modelo analítico.

AUDITORIA_TARIFA

Se crea en la Entrega 1 y permanece vacía. Es el destino del disparador de sentencia exigido en la Entrega 2 (quién, cuándo, valor anterior y nuevo). Crearla ahora evita modificar el DDL más adelante.

4.5 Normalización

El modelo está en tercera forma normal. Las excepciones son deliberadas y se documentan:

- `RESERVA.valor_alojamiento` y `valor_servicios` son redundancia calculada aceptada a conciencia: recalcular el total de 25.000 reservas explotando sus noches cuesta segundos, mientras que el mostrador necesita el total en milisegundos. En la Entrega 2 el paquete será el único punto que la mantenga.
- `RESERVA.valor_total` es una columna virtual (`GENERATED ALWAYS AS`). No se almacena, se calcula al leer, y por construcción no puede desincronizarse.
- `RESERVA_SERVICIO.valor_total` (`cantidad × valor_unitario`) también es virtual.
- `RESERVA_SERVICIO.valor_unitario` no es redundante frente a `SERVICIO.precio_base`: guarda el precio histórico del día en que se prestó el servicio. Si mañana sube el tour del café, las facturas del año pasado no deben cambiar.

4.6 Qué válida el motor y qué queda para PL/SQL

Regla	Cómo se impone	Por qué
<code>fecha_checkout > fecha_checkin</code>	CHECK	Compara dos columnas de la misma fila
Calificación entre 1 y 5	CHECK	Constante
Habitación única dentro de la reserva	UNIQUE	Clave compuesta
Una reseña por reserva	UNIQUE sobre la FK	Impone la cardinalidad máxima
Rol coherente con alojamiento	CHECK compuesto	Involucra solo la misma fila

Regla	Cómo se impone	Por qué
Fecha de nacimiento no futura	PL/SQL (Entrega 2)	SYSDATE no es determinista y Oracle prohíbe funciones no deterministas en un CHECK
La habitación pertenece al alojamiento de la reserva	PL/SQL (Entrega 2)	Involucra tres tablas
Sin solapamiento de reservas	Disparador de fila (Entrega 2)	Involucra otras filas de la misma tabla
Capacidad no excedida	PL/SQL (Entrega 2)	Requiere sumar las filas hijas
Temporadas sin superposición	Disparador (Entrega 2)	Oracle no ofrece restricción de exclusión de rangos
Reserva con al menos una habitación	PL/SQL (Entrega 2)	Cardinalidad mínima; ninguna FK la expresa

5. Diccionario de datos

Se describen las 17 tablas creadas. El conteo de restricciones efectivamente declaradas en la base es: 17 claves primarias, 17 claves foráneas, 16 restricciones UNIQUE y 141 restricciones CHECK y NOT NULL.

5.1 MUNICIPIO

Propósito: los 12 municipios del departamento del Quindío, con su código DANE real.

Columna	Tipo	Restricción	Descripción
id_municipio	NUMBER(4)	PK, DEFAULT secuencia	Identificador
nombre	VARCHAR2(60)	NOT NULL, UNIQUE	Nombre oficial del municipio
codigo_dane	CHAR(5)	NOT NULL, UNIQUE	Código DANE real (63xxx)
altitud_msnm	NUMBER(5)	NOT NULL, CHECK 500–3500	Altitud; relevante en turismo de montaña
poblacion	NUMBER(8)	CHECK > 0	Población aproximada

5.2 TIPO_ALOJAMIENTO

Propósito: clasificación de la oferta. Valores cargados: FINCA_CAFETERA, HOTEL, GLAMPING, HOSTAL, CABANA, APARTAHOTEL.

Columna	Tipo	Restricción	Descripción
id_tipo	NUMBER(3)	PK	Identificador
nombre	VARCHAR2(30)	NOT NULL, UNIQUE	Tipo de alojamiento
descripcion	VARCHAR2(200)	—	Detalle descriptivo

5.3 CATEGORIA_HABITACION [D]

Propósito: categorías de habitación. Valores cargados: ESTANDAR, SUPERIOR, SUITE.

Columna	Tipo	Restricción	Descripción
codigo	VARCHAR2(20)	PK	Código de categoría
descripcion	VARCHAR2(120)	NOT NULL	Detalle
factor_precio	NUMBER(4,2)	NOT NULL, CHECK > 0	Multiplicador sobre el precio base

5.4 ALOJAMIENTO

Propósito: establecimientos de hospedaje registrados en la plataforma.

Columna	Tipo	Restricción	Descripción
id_alojamiento	NUMBER(6)	PK	Identificador
id_municipio	NUMBER(4)	FK, NOT NULL	Ubicación
id_tipo	NUMBER(3)	FK, NOT NULL	Tipo de alojamiento
nombre	VARCHAR2(120)	NOT NULL, UNIQUE con municipio	Nombre comercial
direccion	VARCHAR2(150)	NOT NULL	Dirección física

Columna	Tipo	Restricción	Descripción
telefono	VARCHAR2(20)	—	Contacto
email	VARCHAR2(120)	CHECK contiene @	Correo de contacto
estrellas	NUMBER(1)	CHECK 1–5	Categoría turística
fecha_registro	DATE	NOT NULL, DEFAULT	Alta en la plataforma
activo	CHAR(1)	NOT NULL, CHECK S/N	Baja lógica: no se borra porque hay histórico

5.5 HABITACION

Propósito: unidades alojables. Es el recurso escaso del sistema y el objeto de bloqueo en el experimento de concurrencia de la Entrega 3.

Columna	Tipo	Restricción	Descripción
id_habitacion	NUMBER(6)	PK	Identificador
id_alojamiento	NUMBER(6)	FK, NOT NULL	Alojamiento propietario
numero	VARCHAR2(10)	NOT NULL, UNIQUE con alojamiento	Número visible de la habitación
categoria	VARCHAR2(20)	FK a CATEGORIA_HABITACION	Determina la tarifa aplicable
capacidad	NUMBER(2)	NOT NULL, CHECK 1–10	Huéspedes máximos
activa	CHAR(1)	NOT NULL, CHECK S/N	Permite retirarla de servicio sin borrarla

5.6 TEMPORADA

Propósito: rangos de fechas con comportamiento comercial homogéneo. Es un rango del calendario, no un atributo de la reserva.

Columna	Tipo	Restricción	Descripción
id_temporada	NUMBER(5)	PK	Identificador
nombre	VARCHAR2(60)	NOT NULL, UNIQUE	Etiqueta legible
tipo	VARCHAR2(5)	CHECK ALTA/MEDIA/BAJA	Clasificación comercial
fecha_inicio	DATE	NOT NULL, UNIQUE, CHECK sin hora	Inicio del rango
fecha_fin	DATE	NOT NULL, CHECK >= inicio	Fin del rango (cerrado)
factor	NUMBER(4,2)	CHECK entre 0,5 y 5	Multiplicador estacional

5.7 TARIFA

Propósito: precio por noche de la terna (alojamiento, categoría, temporada).

Columna	Tipo	Restricción	Descripción
id_tarifa	NUMBER(8)	PK	Identificador
id_alojamiento	NUMBER(6)	FK, NOT NULL	Alojamiento
categoria	VARCHAR2(20)	FK, NOT NULL	Categoría de habitación
id_temporada	NUMBER(5)	FK, NOT NULL	Temporada

Columna	Tipo	Restricción	Descripción
valor_noche	NUMBER(10,2)	NOT NULL, CHECK > 0	Precio en pesos por noche
—	—	UNIQUE (alojamiento, categoría, temporada)	Clave de negocio y camino de acceso de fn_valor_estadia

5.8 DIM_TIEMPO [D]

Propósito: calendario del 1 de enero de 2024 al 31 de diciembre de 2026. Cada día apunta a exactamente una temporada. Filas cargadas: 1.096 (incluye el bisiesto de 2024).

Columna	Tipo	Restricción	Descripción
fecha	DATE	PK, CHECK sin hora	Día del calendario
anio, mes, dia	NUMBER	NOT NULL, CHECK de rango	Atributos derivados
anio_mes	CHAR(7)	NOT NULL	Formato YYYY-MM, clave de agrupación mensual
nombre_mes	VARCHAR2(15)	NOT NULL	En español, sin depender del NLS del cliente
trimestre	NUMBER(1)	CHECK 1–4	Trimestre calendario
dia_semana	NUMBER(1)	CHECK 1–7	1 = lunes
es_fin_semana	CHAR(1)	CHECK S/N	Sábado o domingo
id_temporada	NUMBER(5)	FK, NOT NULL	Temporada a la que pertenece el día

5.9 CLIENTE

Propósito: huéspedes registrados. Todos los datos cargados son ficticios generados; no se usaron datos personales reales.

Columna	Tipo	Restricción	Descripción
id_cliente	NUMBER(7)	PK	Identificador
tipo_documento	VARCHAR2(3)	CHECK CC/CE/PAS/TI/NIT	Tipo de documento
numero_documento	VARCHAR2(20)	UNIQUE junto con el tipo	Número de documento
nombres, apellidos	VARCHAR2(60)	NOT NULL	Datos del huésped
email	VARCHAR2(120)	NOT NULL, UNIQUE, CHECK @	Correo de contacto
telefono	VARCHAR2(20)	—	Teléfono
fecha_nacimiento	DATE	CHECK >= 1910-01-01	Ver sección 4.6 sobre SYSDATE
pais, ciudad_origen	VARCHAR2	pais NOT NULL	Origen del turista
fecha_registro	DATE	NOT NULL, DEFAULT	Alta en la plataforma

5.10 RESERVA

Propósito: cabecera del contrato de estadía. Es la tabla central del histórico y reside en TS_TURISMO_HIST.

Columna	Tipo	Restricción	Descripción
id_reserva	NUMBER(8)	PK	Identificador
id_cliente	NUMBER(7)	FK, NOT NULL	Quién reserva

Columna	Tipo	Restricción	Descripción
id_alojamiento	NUMBER(6)	FK, NOT NULL	Dónde se hospeda
fecha_reserva	DATE	NOT NULL, CHECK <= checkin	Cuándo se hizo la reserva
fecha_checkin	DATE	NOT NULL, CHECK sin hora	Entrada
fecha_checkout	DATE	NOT NULL, CHECK > checkin	Salida. Requisito explícito del documento
num_huespedes	NUMBER(3)	CHECK 1–40	Total de personas
estado	VARCHAR2(15)	CHECK de 5 valores	PENDIENTE, CONFIRMADA, CANCELADA, COMPLETADA, NO_SHOW
canal	VARCHAR2(15)	CHECK de 4 valores	WEB, TELEFONO, AGENCIA, PRESENCIAL
valor_alojamiento	NUMBER(12,2)	CHECK >= 0	Suma de las líneas de habitación
valor_servicios	NUMBER(12,2)	CHECK >= 0	Suma de los servicios consumidos
valor_total	NUMBER(12,2)	Columna virtual	Suma de los dos anteriores; no se almacena

5.11 RESERVA_HABITACION

Propósito: entidad asociativa que resuelve la relación N:M entre reservas y habitaciones. Filas cargadas: 28.539 para 25.000 reservas.

Columna	Tipo	Restricción	Descripción
id_reserva_habitacion	NUMBER(9)	PK	Identificador
id_reserva	NUMBER(8)	FK ON DELETE CASCADE	Cabecera
id_habitacion	NUMBER(6)	FK, NOT NULL	Habitación asignada
—	—	UNIQUE (id_reserva, id_habitacion)	Evita repetir la habitación en la misma reserva
huespedes	NUMBER(2)	CHECK 1–10	Personas en esa habitación
valor_noches	NUMBER(12,2)	CHECK >= 0	Valor multi-temporada de esa habitación

5.12 PAGO

Propósito: movimientos de cobro. Admite varios pagos por reserva, lo que permite modelar anticipos. Filas cargadas: 30.232.

Columna	Tipo	Restricción	Descripción
id_pago	NUMBER(9)	PK	Identificador
id_reserva	NUMBER(8)	FK ON DELETE CASCADE	Reserva pagada
fecha_pago	DATE	NOT NULL	Fecha del movimiento
valor	NUMBER(12,2)	CHECK > 0	Importe
medio_pago	VARCHAR2(20)	CHECK de 5 valores	EFFECTIVO, TARJETA_CREDITO, TARJETA_DEBITO, PSE, TRANSFERENCIA
estado	VARCHAR2(15)	CHECK de 3 valores	APROBADO, RECHAZADO, REVERSADO
referencia	VARCHAR2(30)	NOT NULL, UNIQUE	Referencia de la pasarela

5.13 SERVICIO y RESERVA_SERVICIO

SERVICIO es el catálogo de servicios complementarios (15 cargados: tour del café, cabalgata, transporte al aeropuerto, spa y otros), con nombre único, categoría y precio base. RESERVA_SERVICIO registra el consumo: 40.000 líneas cargadas.

Columna (RESERVA_SERVICIO)	Tipo	Restricción	Descripción
id_reserva_servicio	NUMBER(9)	PK	Identificador
id_reserva	NUMBER(8)	FK ON DELETE CASCADE	Reserva asociada
id_servicio	NUMBER(5)	FK, NOT NULL	Servicio prestado
fecha_servicio	DATE	NOT NULL	Debe caer dentro de la estadía
cantidad	NUMBER(4)	CHECK > 0	Unidades consumidas
valor_unitario	NUMBER(10,2)	CHECK > 0	Precio histórico del día de la prestación
valor_total	NUMBER(12,2)	Columna virtual	cantidad × valor_unitario

5.14 RESEÑA

Propósito: valoración posterior a la estadía. Una por reserva. Filas cargadas: 8.957. Las cinco calificaciones separadas son el insumo directo de la consulta 7 (UNPIVOT).

Columna	Tipo	Restricción	Descripción
id_resena	NUMBER(8)	PK	Identificador
id_reserva	NUMBER(8)	FK, UNIQUE	Impone una sola reseña por reserva
fecha_resena	DATE	NOT NULL	Fecha de publicación
calif_general	NUMBER(1)	CHECK 1–5	Calificación global
calif_limpieza	NUMBER(1)	CHECK 1–5	Dimensión limpieza
calif_ubicacion	NUMBER(1)	CHECK 1–5	Dimensión ubicación
calif_servicio	NUMBER(1)	CHECK 1–5	Dimensión servicio
calif_precio	NUMBER(1)	CHECK 1–5	Dimensión relación precio-valor
comentario	VARCHAR2(500)	—	Texto libre

5.15 USUARIO_SISTEMA

Propósito: usuarios de la plataforma. Los cuatro roles exigidos en la Entrega 3 ya son valores válidos del modelo. Filas cargadas: 125.

Columna	Tipo	Restricción	Descripción
id_usuario	NUMBER(6)	PK	Identificador
username	VARCHAR2(30)	NOT NULL, UNIQUE	Nombre de acceso
nombre_completo	VARCHAR2(120)	NOT NULL	Nombre del usuario
email	VARCHAR2(120)	NOT NULL, UNIQUE	Correo
rol	VARCHAR2(20)	CHECK de 4 valores	RECEPCION, ADMIN_ALOJAMIENTO, GERENTE, AUDITOR

Columna	Tipo	Restricción	Descripción
id_alojamiento	NUMBER(6)	FK, condicional	Obligatorio para recepción y administrador; nulo para gerente y auditor
activo	CHAR(1)	NOT NULL, CHECK S/N	Estado del usuario

La condicionalidad del ámbito se impone con un único CHECK compuesto: recepción y administrador de alojamiento pertenecen a un establecimiento concreto, mientras que gerente y auditor son roles transversales al departamento. Esta separación es la base del modelo de privilegios diferenciados de la Entrega 3.

5.16 AUDITORIA_TARIFA [D]

Propósito: registro de cambios sobre TARIFA. Vacía en la Entrega 1; se puebla con el disparador de sentencia de la Entrega 2. Registra la operación, el id de la tarifa, el valor anterior y el nuevo, el usuario de base de datos (DEFAULT USER), la marca de tiempo (SYSTIMESTAMP) y el número de filas afectadas.

6. Tablespace para el histórico de reservas [R]

El documento exige un tablespace propio para el histórico de reservas, con la justificación de por qué separarlo. Se crearon tres tablespaces, no uno, y la distinción es parte de la justificación.

Tablespace	Contenido	Tamaño inicial	Crecimiento
TS_TURISMO_DAT	Catálogo y dimensiones: 12 tablas	100 MB	AUTOEXTEND 25 MB, máx. 2 GB
TS_TURISMO_HIST	Histórico transaccional: 6 tablas	300 MB	AUTOEXTEND 50 MB, máx. 6 GB
TS_TURISMO_IDX	Todos los índices del esquema	100 MB	AUTOEXTEND 25 MB, máx. 3 GB

6.1 Qué se ubica en el histórico

RESERVA, RESERVA_HABITACION, PAGO, RESERVA_SERVICIO, RESENA y AUDITORIA_TARIFA residen en TS_TURISMO_HIST.

6.2 Por qué estas tablas son "histórico"

Las seis comparten tres propiedades que ninguna tabla de catálogo tiene:

- Solo crecen. Un municipio se crea una vez; las reservas se acumulan indefinidamente. Concentran más del 95 % de las filas del sistema y prácticamente el 100 % de su crecimiento futuro.
- Son inmutables una vez cerradas. Una reserva de 2024 ya completada no vuelve a modificarse: son datos de solo lectura fuera de su ventana activa.
- Su patrón de acceso es distinto. Se leen en barridos analíticos por rango de fechas, mientras que el catálogo se lee por clave y cabe holgadamente en caché.

6.3 Ventajas de la separación

- Administración independiente: el histórico puede marcarse como READ ONLY por periodos cerrados, respaldarse con otra frecuencia y crecer con su propia política de AUTOEXTEND sin arrastrar al catálogo.
- Contención del crecimiento: un error en el generador de datos llena TS_TURISMO_HIST sin comprometer el resto del esquema.
- Respaldo y recuperación por tablespace: RMAN opera a nivel de tablespace, de modo que se puede recuperar el histórico sin tocar el catálogo y viceversa.
- Medición: consultando USER_SEGMENTS filtrado por tablespace se puede cuantificar exactamente cuánto ocupan los datos frente a los índices, que viven en TS_TURISMO_IDX. Esto será relevante para evaluar el costo en disco de los índices de la Entrega 3.
- Fragmentación: datos e índices crecen de forma distinta; separarlos evita que sus extensiones se intercalen.

6.4 Requisitos previos de permisos

Los comandos de creación no funcionan con cualquier usuario y conviene tenerlo claro:

- CREATE TABLESPACE y CREATE USER se ejecutan como SYSDBA y dentro del PDB XEPDB1, no en la raíz CDB\$ROOT. En la raíz, un usuario nuevo tendría que llamarse C##...

- El privilegio CREATE TABLE no basta. Sin QUOTA sobre el tablespace, el usuario recibe ORA-01950: no privileges on tablespace. Por eso se otorga QUOTA UNLIMITED sobre los tres. En la verificación, las cuotas aparecen con valor -1, que es como Oracle representa UNLIMITED.
- La ruta del datafile debe existir en el servidor. Se verificó con SELECT name FROM v\$datafile, que en este entorno devuelve /opt/oracle/oradata/XE/XEPDB1/.

6.5 Limitaciones de Oracle XE 21c

Oracle XE limita el volumen de datos de usuario a 12 GB, la SGA a 2 GB y el uso a 2 hilos de CPU. El proyecto completo ocupa unos pocos cientos de megabytes, muy por debajo del límite, pero la restricción existe y conviene mencionarla. Tampoco incluye Real Application Clusters ni Active Data Guard.

Respecto al particionamiento: el modelo no depende de él, y la separación por tablespace es suficiente para lo que exige el documento. Si la opción estuviera disponible en la instalación (verificable con SELECT * FROM v\$option WHERE parameter = 'Partitioning'), la extensión natural sería particionar RESERVA por rango de fecha_checkin, con cada partición anual en su propio tablespace y las antiguas en modo solo lectura.

TABLESPACE_NAME	STATUS	CONTENTS	EXTENT_MAN	SEGMENT
TS_TURISMO_DAT	ONLINE	PERMANENT	LOCAL	AUTO
TS_TURISMO_HIST	ONLINE	PERMANENT	LOCAL	AUTO
TS_TURISMO_IDX	ONLINE	PERMANENT	LOCAL	AUTO

TABLESPACE_NAME	MAX_BYTES
TS_TURISMO_HIST	-1
TS_TURISMO_IDX	-1
TS_TURISMO_DAT	-1

Figura 2. Tablespaces creados y cuotas asignadas al usuario TURISMO_UQ.

7. Generación de datos

El documento exige generar los datos con PL/SQL y DBMS_RANDOM, o con Mockaroo. Se optó por PL/SQL, lo que permite garantizar la coherencia de los datos por construcción.

7.1 Principios de la generación

- Reproducibilidad: cada bloque fija una semilla con DBMS_RANDOM.SEED, de modo que dos ejecuciones producen los mismos datos.
- Coherencia por construcción: todas las claves foráneas se toman de arreglos cargados desde las tablas padre con BULK COLLECT, nunca de un número aleatorio esperando que exista.
- Cero solapamientos: un mapa de ocupación en memoria, implementado como un arreglo asociativo indexado por posición de habitación y día, marca cada noche vendida. Antes de asignar una habitación se verifica que todas sus noches estén libres. Esto evita 25.000 consultas de disponibilidad contra la base y garantiza que el disparador de la Entrega 2 sea coherente con los datos ya cargados.
- Estacionalidad real: el 55 % de los check-in se sortean entre días de temporada ALTA o MEDIA. Sin ese sesgo, las consultas 1, 2, 4 y 8 mostrarían una línea plana sin nada que analizar.
- Cobertura garantizada: los primeros 12 alojamientos se asignan uno por municipio, y los 48 restantes por ruleta ponderada según peso turístico. Sin esa garantía, un municipio de peso bajo podía quedar sin oferta y desaparecer de las consultas de análisis.

7.2 Construcción de las temporadas

El documento indica que la demanda se dispara en Semana Santa, mitad de año, los puentes festivos y diciembre-enero. A partir de esa descripción se definieron las siguientes reglas de clasificación [D]:

Tipo	Regla aplicada	Factor
ALTA	Del 15 de diciembre al 15 de enero; Semana Santa con la fecha real de cada año (2024-03-24 a 03-31; 2025-04-13 a 04-20; 2026-03-29 a 04-05); del 15 de junio al 15 de julio	1,60
MEDIA	Bloque de viernes a lunes de la segunda semana de cada mes, como aproximación reproducible de los puentes festivos colombianos	1,25
BAJA	Todo el resto del calendario	1,00

El algoritmo clasifica día por día en un arreglo asociativo y luego comprime los días consecutivos del mismo tipo en filas de TEMPORADA. Este procedimiento garantiza por construcción que las temporadas son disjuntas y cubren el rango completo sin huecos, que es exactamente la propiedad de la que depende el cálculo correcto del valor de una estadía.

Una nota técnica que conviene explicar: el día de la semana se calcula con MOD(fecha - DATE '1900-01-01', 7) en lugar de TO_CHAR(fecha,'D') o NEXT_DAY, porque esas dos funciones dependen de la configuración NLS del cliente y harían el script no reproducible entre equipos.

Resultado: 72 temporadas que cubren los 1.096 días de 2024 a 2026, distribuidos en 70,6 % baja, 19,4 % alta y 9,9 % media.

7.3 Cálculo del valor de las estadías

Aquí se resuelve en SQL puro el problema que el documento señala como el más difícil. Cada noche de la estadía es una fila de DIM_TIEMPO; cada fila conoce su temporada; con la terna (alojamiento, categoría, temporada) se obtiene la tarifa de esa noche. Sumar por línea de reserva da el valor correcto aunque la estadía cruce dos, tres o más temporadas.

```

MERGE INTO reserva_habitacion rh
USING (
    SELECT rh2.id_reserva_habitacion AS id, SUM(t.valor_noche) AS valor
    FROM reserva_habitacion rh2
    JOIN reserva      r ON r.id_reserva      = rh2.id_reserva
    JOIN habitacion   h ON h.id_habitacion   = rh2.id_habitacion
    JOIN dim_tiempo   d ON d.fecha >= r.fecha_checkin
                        AND d.fecha < r.fecha_checkout
    JOIN tarifa       t ON t.id_alojamiento = h.id_alojamiento
                        AND t.categoria     = h.categoria
                        AND t.id_temporada  = d.id_temporada
    GROUP BY rh2.id_reserva_habitacion
) x ON (rh.id_reserva_habitacion = x.id)
WHEN MATCHED THEN UPDATE SET rh.valor_noches = x.valor;

```

La función fn_valor_estadia de la Entrega 2 será esta misma lógica encapsulada en PL/SQL. El modelo no requiere ningún cambio para soportarla.

7.4 Volúmenes obtenidos

Comparación entre lo exigido por el documento y lo efectivamente cargado:

Tabla	Exigido	Cargado	Estado
MUNICIPIO	12	12	OK
ALOJAMIENTO	60	60	OK
HABITACION	400	400	OK
CLIENTE	3.000	3.000	OK
RESERVA	25.000	25.000	OK
RESERVA_SERVICIO	40.000	40.000	OK
RESERVA_HABITACION	—	28.539	OK
TEMPORADA	—	72	OK
TARIFA	—	12.960	OK
DIM_TIEMPO	—	1.096	OK
PAGO	—	30.232	OK
RESENA	—	8.957	OK
USUARIO_SISTEMA	—	125	OK

Distribución de las reservas en el periodo exigido 2024–2026: 8.276 en 2024 (33,1 %), 8.202 en 2025 (32,8 %) y 8.522 en 2026 (34,1 %).

TABLA	FILAS
MUNICIPIO	12
TIPO_ALOJAMIENTO	6
CATEGORIA_HABITACION	3
ALOJAMIENTO	60
HABITACION	400
TEMPORADA	72
DIM_TIEMPO	1096
TARIFA	12960
CLIENTE	3000
SERVICIO	15
USUARIO_SISTEMA	125
TABLA	FILAS
RESERVA	25000
RESERVA_HABITACION	28539
PAGO	30232
RESERVA_SERVICIO	40000
RESENA	8957

Figura 3. Resumen de la carga masiva de datos.

8. Las ocho consultas de análisis

A continuación se documenta cada consulta con su pregunta de negocio, la técnica SQL empleada, la justificación de esa técnica y el resultado obtenido. Todas se apoyan en la vista V_NOCHE_VENDIDA, que explota cada estadía en sus noches y asigna a cada una la tarifa de su temporada, de modo que los resultados de las ocho son mutuamente consistentes.

Consulta 1 · Ocupación por municipio y mes (PIVOT) [R]

Aspecto	Detalle
Pregunta de negocio	¿Cómo se comporta la ocupación de cada municipio a lo largo del año?
Definición de ocupación [D]	Noches-habitación vendidas ÷ noches-habitación disponibles, donde el denominador es habitaciones activas del municipio × días del mes
Periodo analizado	Año 2025, por ser el único año completo del histórico
Técnica	PIVOT sobre los 12 meses
Por qué PIVOT	La consulta agrupada natural devuelve 12 municipios × 12 meses = 144 filas verticales, ilegibles para comparar estacionalidad. PIVOT gira los meses a columnas y produce una matriz que se lee de un vistazo. Es una transformación de presentación: la función MAX opera sobre un valor único por celda
Resultado obtenido	12 filas (una por municipio) × 12 columnas de meses, con picos visibles en junio-julio y diciembre, y valles en febrero, mayo y septiembre

```

WITH ventas AS (
    SELECT nv.id_municipio, nv.mes, COUNT(*) AS noches
    FROM v_noche_vendida nv
    WHERE nv.anio = 2025
    GROUP BY nv.id_municipio, nv.mes
),
disponibles AS (
    SELECT iv.id_municipio, iv.mes, SUM(iv.noches_disponibles) AS noches
    FROM v_inventario_noche iv
    WHERE iv.anio = 2025
    GROUP BY iv.id_municipio, iv.mes
),
ocupacion AS (
    SELECT m.nombre AS municipio,
           dsp.mes,
           ROUND(100 * NVL(vnd.noches, 0) / dsp.noches, 1) AS pct
    FROM disponibles dsp
    JOIN municipio m ON m.id_municipio = dsp.id_municipio
    LEFT JOIN ventas vnd ON vnd.id_municipio = dsp.id_municipio
                        AND vnd.mes = dsp.mes
)

SELECT *
FROM ocupacion
PIVOT (MAX(pct) FOR mes IN (1 AS ENE, 2 AS FEB, 3 AS MAR, 4 AS ABR,
                           5 AS MAY, 6 AS JUN, 7 AS JUL, 8 AS AGO,
                           9 AS SEP, 10 AS OCT, 11 AS NOV, 12 AS DIC))

ORDER BY municipio;

```

MUNICIPIO	ENE	FEB	MAR	ABR	MAY	JUN	JUL	AGO	SEP	OCT	NOV	DIC
1 Armenia	19,6	16,1	14,6	17,9	12,7	24,2	21,5	11,1	9,6	12,1	12,1	22,4
2 Buenavista	22,9	8,6	6,9	13,7	10,5	21,7	18,7	7,3	9,1	11,9	14,7	20,1
3 Calarcá	25,2	13,5	14,8	18,8	14,6	30,4	26,7	7,8	16,3	10,9	15,2	27
4 Circasia	21,9	12,8	11,1	16	9,3	24,4	24,9	10,4	14,4	9	12,5	28,2
5 Córdoba	25,1	13,3	12,4	5,5	15,4	34,5	20,7	6,2	13,3	7,1	13,1	24,9
6 Filandia	22	12,9	11,7	17,3	11,8	26,2	22,2	9,6	10,4	9,8	12,9	20,6
7 Génova	14	9,2	8,3	18,1	8,1	33,3	29,3	10,5	19,2	15,9	12,5	30,6

Figura 4. Consulta 1: ocupación porcentual por municipio y mes mediante PIVOT.

Consulta 2 · Ingresos con ROLLUP y GROUPING [R]

Aspecto	Detalle
Pregunta de negocio	¿De dónde viene el ingreso, con subtotales por municipio y por tipo de alojamiento?
Definición de ingreso [D]	Ingreso devengado: suma de las tarifas noche a noche de reservas confirmadas o completadas. No se usan los pagos, porque un pago no sabe a qué temporada pertenece la noche que financia
Técnica	GROUP BY ROLLUP (municipio, tipo, temporada) con GROUPING y GROUPING_ID

Aspecto	Detalle
Por qué ROLLUP y no CUBE	Las tres columnas forman una jerarquía natural (municipio > tipo > temporada) y ROLLUP produce exactamente n+1 niveles, que es como se lee un estado de resultados. CUBE generaría 2 elevado a n combinaciones, entre ellas subtotales sin sentido jerárquico, multiplicando las filas sin responder mejor la pregunta
Para qué GROUPING	Devuelve 1 cuando la fila es un subtotal respecto de esa columna. Sin él es imposible distinguir un subtotal (que muestra NULL en la columna agregada) de un NULL real de los datos
Para qué GROUPING_ID	Codifica en un solo número el mapa de bits de las columnas agregadas, lo que permite ordenar por nivel: 0 detalle, 1 subtotal por tipo, 3 subtotal por municipio, 7 total general
Resultado obtenido	165 filas con detalle, subtotales por tipo, subtotales por municipio y total general etiquetados. Total departamental 2025: 24.342 noches y 7.791.164.000 pesos

```

SELECT CASE WHEN GROUPING(m.nombre) = 1 THEN '>> TOTAL GENERAL'
           ELSE m.nombre END                               AS municipio,
       CASE WHEN GROUPING(ta.nombre) = 1
            AND GROUPING(m.nombre) = 0 THEN '> Subtotal municipio'
            WHEN GROUPING(ta.nombre) = 1 THEN ' '
            ELSE ta.nombre END                             AS tipo_alojamiento,
       CASE WHEN GROUPING(t.tipo) = 1
            AND GROUPING(ta.nombre) = 0 THEN '> Subtotal tipo'
            WHEN GROUPING(t.tipo) = 1 THEN ' '
            ELSE t.tipo END                                 AS temporada,
       COUNT(*)                                            AS noches,
       SUM(nv.valor_noche)                                AS ingreso,
       -- GROUPING_ID codifica en binario qué columnas están agregadas:
       -- 0 = detalle · 1 = subtotal por tipo · 3 = subtotal por municipio
       -- 7 = total general

       GROUPING_ID(m.nombre, ta.nombre, t.tipo)           AS nivel
FROM v_noche_vendida nv
JOIN municipio      m ON m.id_municipio = nv.id_municipio
JOIN tipo_alojamiento ta ON ta.id_tipo = nv.id_tipo
JOIN temporada     t ON t.id_temporada = nv.id_temporada
WHERE nv.anio = 2025
GROUP BY ROLLUP (m.nombre, ta.nombre, t.tipo)
ORDER BY GROUPING_ID(m.nombre, ta.nombre, t.tipo),
         m.nombre, ta.nombre, t.tipo;

```

	MUNICIPIO	TIPO_ALOJAMIENTO	TEMPORADA	NOCHES	INGRESO	NIVEL
127	Salento	HOTEL	ALTA	208	78992000	0
128	Salento	HOTEL	BAJA	153	35018000	0
129	Salento	HOTEL	MEDIA	70	20262000	0
130	Armenia	APARTAHOTEL	> Subtot...	1349	259379000	1
131	Armenia	CABANA	> Subtot...	1273	339710000	1
132	Armenia	FINCA_CAFETERA	> Subtot...	301	127121000	1
133	Armenia	GLAMPING	> Subtot...	384	161570000	1

Figura 5. Consulta 2: ingresos con ROLLUP, distinguiendo detalle, subtotales y total general.

Consulta 3 · Top 3 alojamientos por municipio (RANK) [R]

Aspecto	Detalle
Pregunta de negocio	¿Qué alojamientos lideran dentro de cada municipio?
Técnica	RANK() OVER (PARTITION BY id_municipio ORDER BY ingreso DESC)
Por qué no FETCH FIRST 3 ROWS	Devolvería los tres mejores del departamento, que es otra pregunta. PARTITION BY es lo que reinicia el conteo en cada municipio
Por qué RANK y no ROW_NUMBER	ROW_NUMBER rompería un empate de forma arbitraria y dejaría fuera a un alojamiento con el mismo ingreso que el tercero. RANK respeta el empate; el precio aceptado es que un municipio empatado puede devolver más de tres filas
Detalle adicional	Se incluye RATIO_TO_REPORT para mostrar qué porcentaje del ingreso del municipio aporta cada alojamiento
Resultado obtenido	29 filas: tres por municipio, salvo Córdoba, Génova y Pijao (un solo alojamiento, que concentra el 100 %) y Buenavista (dos alojamientos)

```

WITH ingresos AS (
    SELECT nv.id_municipio,
           nv.id_alojamiento,
           SUM(nv.valor_noche) AS ingreso,
           COUNT(*)           AS noches
    FROM v_noche_vendida nv
    WHERE nv.anio = 2025
    GROUP BY nv.id_municipio, nv.id_alojamiento
),
ranking AS (
    SELECT i.*,
           RANK() OVER (PARTITION BY i.id_municipio
                        ORDER BY i.ingreso DESC) AS puesto,
           ROUND(100 * RATIO_TO_REPORT(i.ingreso)
                OVER (PARTITION BY i.id_municipio), 1) AS pct_municipio
    FROM ingresos i
)
SELECT m.nombre AS municipio,
       r.puesto,
       a.nombre AS alojamiento,
       ta.nombre AS tipo,
       r.noches,
       r.ingreso,
       r.pct_municipio AS pct_del_municipio
FROM ranking      r
JOIN municipio    m ON m.id_municipio = r.id_municipio
JOIN alojamiento  a ON a.id_alojamiento = r.id_alojamiento
JOIN tipo_alojamiento ta ON ta.id_tipo = a.id_tipo
WHERE r.puesto <= 3
ORDER BY m.nombre, r.puesto;

```

	MUNICIPIO	PUESTO	ALOJAMIENTO	TIPO	NOCHES	INGRESO	PCT_DEL_MUNICIPIO
1	Armenia	1	Mirador La Esperanza 15	HOTEL	417	185026000	13,5
2	Armenia	2	Cabañas Alto Bonito 1	GLAMPING	384	161570000	11,8
3	Armenia	3	Villa La Cascada 21	CABANA	396	159763000	11,7
4	Buenavista	1	Refugio La Cascada 2	APARTAHOTEL	414	176915000	49,2
5	Buenavista	2	Cabañas El Ocaso 59	APARTAHOTEL	402	66814000	18,6
6	Buenavista	3	Finca La Cascada 38	FINCA_CAFETERA	394	61396000	17,1
7	Calarcá	1	Hotel El Mirador 3	HOSTAL	497	218163000	25,3

Figura 6. Consulta 3: ranking de los tres mejores alojamientos dentro de cada municipio.

Consulta 4 · Variación de ingresos mes contra mes (LAG) [R]

Aspecto	Detalle
Pregunta de negocio	¿Cómo evoluciona el ingreso mes a mes y cuál es la variación porcentual?
Técnica	LAG(ingreso, 1) OVER (ORDER BY anio_mes), más LAG(ingreso, 12) para la comparación interanual
Por qué LAG	Trae el valor de la fila anterior sin necesidad de auto-unir la tabla consigo misma desplazada un mes

Aspecto	Detalle
Manejo del primer mes	LAG devuelve NULL porque no existe mes anterior. Se deja NULL en la diferencia: poner 0 afirmaría que no hubo variación, lo cual sería falso. El porcentaje se protege con NULLIF para evitar la división por cero
Resultado obtenido	36 filas (los 36 meses del histórico). La estacionalidad es nítida: diciembre de 2026 crece 233,6 % sobre noviembre, y agosto cae sistemáticamente entre 64 % y 65 % respecto de julio

```

WITH mensual AS (
    SELECT nv.anio_mes,
           SUM(nv.valor_noche) AS ingreso
    FROM v_noche_venta nv
    GROUP BY nv.anio_mes
)
SELECT anio_mes,
       ingreso,
       LAG(ingreso, 1) OVER (ORDER BY anio_mes) AS ingreso_mes_anterior,
       ingreso - LAG(ingreso, 1) OVER (ORDER BY anio_mes) AS diferencia,
       ROUND(100 * (ingreso - LAG(ingreso, 1) OVER (ORDER BY anio_mes))
            / NULLIF(LAG(ingreso, 1) OVER (ORDER BY anio_mes), 0), 1)
            AS variacion_pct,
       -- comparación interanual: mismo mes del año anterior (12 posiciones atrás)
       LAG(ingreso, 12) OVER (ORDER BY anio_mes) AS ingreso_anio_anterior
FROM mensual
ORDER BY anio_mes;

```

	ANIO_MES	INGRESO	INGRESO_MES_ANTERIOR	DIFERENCIA	VARIACION_PCT	INGRESO_ANIO_ANTERIOR
13	2025-01	983030000	1127396000	-144366000	-12,8	949981000
14	2025-02	390429000	983030000	-592601000	-60,3	420193000
15	2025-03	400286000	390429000	9857000	2,5	796946000
16	2025-04	662938000	400286000	262652000	65,6	468842000
17	2025-05	416382000	662938000	-246556000	-37,2	386729000
18	2025-06	1144852000	416382000	728470000	175	978892000
19	2025-07	1058909000	1144852000	-85943000	-7,5	1050520000
20	2025-08	342195000	1058909000	-716714000	-67,7	367416000
21	2025-09	407813000	342195000	65618000	19,2	361451000
22	2025-10	371480000	407813000	-36333000	-8,9	406622000

Figura 7. Consulta 4: variación mes contra mes con LAG, incluyendo comparación interanual.

Consulta 5 · Consulta parametrizada con variables de enlace [R]

Aspecto	Detalle
Pregunta de negocio	¿Cuánto se facturó entre dos fechas cualesquiera, desglosado por municipio y tipo?
Técnica	Variables de enlace declaradas con VARIABLE y asignadas en un bloque PL/SQL
Por qué variables de enlace	El plan de ejecución se reutiliza desde la caché, con una sola versión del cursor en lugar de una por cada par de fechas; se elimina la posibilidad de inyección SQL; y se reduce el hard parse, que en cargas OLTP es el principal consumidor de CPU
Detalle de sintaxis	En SQL*Plus y SQL Developer el comando VARIABLE no admite el tipo DATE. Se declaran como VARCHAR2 y se convierten con TO_DATE dentro de la consulta, con máscara explícita para no depender del NLS_DATE_FORMAT del cliente
Rango de prueba	Del 1 de junio al 31 de julio de 2025

Aspecto	Detalle
Resultado obtenido	38 combinaciones de municipio y tipo de alojamiento ordenadas por ingreso

```

VARIABLE p_fecha_ini VARCHAR2(10)
VARIABLE p_fecha_fin VARCHAR2(10)

BEGIN
    :p_fecha_ini := '2025-06-01';
    :p_fecha_fin := '2025-07-31';
END;
/

SELECT m.nombre AS municipio,
       ta.nombre AS tipo_alojamiento,
       COUNT(DISTINCT nv.id reserva) AS reservas,
       COUNT(*) AS noches_vendidas,
       SUM(nv.valor_noche) AS ingreso,
       ROUND(AVG(nv.valor_noche)) AS tarifa_media
FROM v_noche_vendida nv
JOIN municipio m ON m.id_municipio = nv.id_municipio
JOIN tipo_alojamiento ta ON ta.id_tipo = nv.id_tipo
WHERE nv.fecha BETWEEN TO_DATE(:p_fecha_ini, 'YYYY-MM-DD')
                  AND TO_DATE(:p_fecha_fin, 'YYYY-MM-DD')
GROUP BY m.nombre, ta.nombre
ORDER BY ingreso DESC;

```

Figura 8. Consulta 5: consulta parametrizada con variables de enlace.

Consulta 6 · Vista materializada de ocupación mensual [R]

Aspecto	Detalle
Pregunta de negocio	¿Cuál es la ocupación, el ingreso y la tarifa media de cada municipio en cada mes?
Objeto creado	MV_OCUPACION_MENSUAL, con 432 filas (12 municipios × 36 meses)
Política elegida	REFRESH COMPLETE ON DEMAND, programada a las 02:00 mediante START WITH y NEXT
Resultado obtenido	Estado FRESH y VALID. La consulta de demostración devuelve el ranking mensual de municipios por ingreso

Justificación de la política de refresco

El documento exige justificar la política y no elegirla automáticamente. Se evaluaron tres alternativas:

- ON COMMIT con FAST: descartada. Obligaría a Oracle a mantener la vista materializada dentro de cada transacción de reserva. La plataforma es OLTP en el mostrador, donde recepción crea reservas todo el día; penalizar cada INSERT para acelerar un informe que se consulta unas pocas veces al día es un mal reparto del costo, y además serializaría los COMMIT sobre la misma fila agregada.

- FAST ON DEMAND: no es viable técnicamente en este caso. El refresco rápido de una vista materializada con agregados exige MATERIALIZED VIEW LOG en todas las tablas base, con ROWID, SEQUENCE e INCLUDING NEW VALUES. Además, Oracle no admite refresco rápido cuando la definición incluye una unión por rango como d.fecha >= r.fecha_checkin AND d.fecha < r.fecha_checkout, que no es un equi-join. Es verificable con DBMS_MVIEW.EXPLAIN_MVIEW.
- COMPLETE ON DEMAND (elegida): el recálculo completo sobre este volumen toma unos pocos segundos y se programa a las 02:00, cuando no hay operación. La ocupación del mes es un indicador de gestión, no un dato de tiempo real: un desfase de horas es irrelevante para la decisión que soporta. Se deja ON DEMAND explícito para poder forzar el refresco con DBMS_MVIEW.REFRESH durante la sustentación.

Se añadió ENABLE QUERY REWRITE, que permite al optimizador redirigir automáticamente hacia la vista materializada consultas equivalentes escritas contra las tablas base, sin modificar el código de la aplicación.

<pre> SELECT m.nombre AS municipio, v.anio_mes, v.reservas, v.noches_vendidas, v.ingreso_alojamiento, v.tarifa_media_noche, RANK() OVER (PARTITION BY v.anio_mes ORDER BY v.ingreso_alojamiento DESC) AS puesto_del_mes FROM mv_ocupacion_mensual v JOIN municipio m ON m.id_municipio = v.id_municipio WHERE v.anio = 2025 AND v.mes IN (7, 12) ORDER BY v.anio_mes, puesto_del_mes; </pre>							
❖ MUNICIPIO	❖ ANIO_MES	❖ RESERVAS	❖ NOCHES_VENDIDAS	❖ INGRESO_ALOJAMIENTO	❖ TARIFA_MEDIA_NOCHE	❖ PUESTO_DEL_MES	
1 Filandia	2025-07	186	585	186944000	319562,39	1	
2 Armenia	2025-07	166	500	178198000	356396	2	
3 Calarcá	2025-07	76	248	118434000	477556,45	3	
4 Circasia	2025-07	96	309	116708000	377695,79	4	
5 Quimbaya	2025-07	91	284	109450000	385387,32	5	
6 Montenegro	2025-07	77	243	89878000	369868,31	6	
7 Salento	2025-07	83	243	79911000	328851,85	7	
8 Buenavista	2025-07	62	180	52257000	290316,67	8	

Figura 9. Consulta 6: vista materializada, su estado de refresco y consulta de demostración.

Consulta 7 · Perfil de calificaciones (UNPIVOT) [R]

Aspecto	Detalle
Pregunta de negocio	¿En qué dimensión concreta de la experiencia falla cada municipio?
Técnica	UNPIVOT sobre las cinco columnas de calificación de RESENA
Por qué UNPIVOT	RESENA guarda cinco calificaciones en cinco columnas. Ese formato ancho es cómodo para capturar y para imponer un CHECK por columna, pero un tablero de calidad necesita el

Aspecto	Detalle
	formato largo: una fila por municipio y dimensión, que es lo único que se puede graficar, ordenar y comparar entre dimensiones. UNPIVOT es la operación inversa del PIVOT de la consulta 1
Detalle técnico	Por defecto UNPIVOT aplica EXCLUDE NULLS. Aquí no hay nulos porque las cinco calificaciones son NOT NULL, pero existe INCLUDE NULLS si se quisiera conservar el hueco
Resultado obtenido	60 filas: 12 municipios × 5 dimensiones. El hallazgo de negocio es consistente en todo el departamento: "Ubicación" es la dimensión mejor calificada (alrededor de 4,38) y "Relación precio-valor" la peor (alrededor de 3,85), algo que el promedio general esconde

```

SELECT municipio, dimension, promedio, resenas
FROM (
    SELECT m.nombre AS municipio,
           COUNT(*) AS resenas,
           ROUND(AVG(re.calif_limpieza), 2) AS limpieza,
           ROUND(AVG(re.calif_ubicacion), 2) AS ubicacion,
           ROUND(AVG(re.calif_servicio), 2) AS servicio,
           ROUND(AVG(re.calif_precio), 2) AS precio,
           ROUND(AVG(re.calif_general), 2) AS general
    FROM resena re
    JOIN reserva r ON r.id_reserva = re.id_reserva
    JOIN alojamiento a ON a.id_alojamiento = r.id_alojamiento
    JOIN municipio m ON m.id_municipio = a.id_municipio
    GROUP BY m.nombre
)
UNPIVOT (promedio FOR dimension IN (limpieza AS 'Limpieza',
                                    ubicacion AS 'Ubicacion',
                                    servicio AS 'Servicio',
                                    precio AS 'Relacion precio-valor',
                                    general AS 'Calificacion general'))
ORDER BY municipio, promedio DESC;

```

	MUNICIPIO	DIMENSION	PROMEDIO	RESENAS
1	Armenia	Ubicacion	4,38	1663
2	Armenia	Limpieza	4,24	1663
3	Armenia	Calificacion general	4,13	1663
4	Armenia	Servicio	4,02	1663
5	Armenia	Relacion precio-valor	3,87	1663
6	Buenavista	Ubicacion	4,37	648
7	Buenavista	Limpieza	4,27	648
8	Buenavista	Calificacion general	4,2	648

Figura 10. Consulta 7: perfil de calificaciones por dimensión mediante UNPIVOT.

Consulta 8 · Consulta libre: ADR, RevPAR e índice de estacionalidad [R]

Aspecto	Detalle
Pregunta de negocio propuesta	¿Qué municipios rinden por precio y cuáles por volumen, y cuáles dependen peligrosamente de la temporada alta?

Aspecto	Detalle
Por qué esta y no otra	Se descartó "qué alojamiento factura más" porque facturar mucho no significa rendir bien: un hotel de 40 habitaciones factura más que un glamping de 6 aun estando medio vacío. Los indicadores estándar del sector separan las dos causas
ADR (Average Daily Rate)	Ingreso ÷ noches vendidas. Responde cuánto se cobra cuando se vende
RevPAR (Revenue Per Available Room)	Ingreso ÷ noches disponibles. Responde cuánto rinde cada habitación exista o no huésped. Equivale a $ADR \times ocupación$
Índice de estacionalidad	Ingreso en temporada alta ÷ ingreso total. Un valor elevado señala un destino frágil, que vive de unas pocas semanas del año
Técnica	Agregación en subconsultas más funciones de ventana sin PARTITION (AVG sobre OVER ()), para comparar cada municipio con el promedio departamental en la misma pasada, sin una segunda consulta
Resultado obtenido	Ranking de los 12 municipios por RevPAR. Pijao lidera con 96.077 pesos pese a tener la oferta más pequeña, mientras Calarcá cierra con 39.592. El RevPAR departamental es 58.140 pesos

```

WITH ingreso_mun AS (
  SELECT nv.id_municipio,
         COUNT(*)           AS noches_vendidas,
         SUM(nv.valor_noche) AS ingreso,
         SUM(CASE WHEN t.tipo = 'ALTA' THEN nv.valor_noche ELSE 0 END) AS ingreso_alta
  FROM v_noche_vendida nv
  JOIN temporada t ON t.id_temporada = nv.id_temporada
  WHERE nv.anio = 2025
  GROUP BY nv.id_municipio
),
inventario_mun AS (
  SELECT id_municipio, SUM(noches_disponibles) AS noches_disponibles
  FROM v_inventario_noche
  WHERE anio = 2025
  GROUP BY id_municipio
)
SELECT m.nombre AS municipio,
       i.noches_vendidas,
       v.noches_disponibles,
       ROUND(100 * i.noches_vendidas / v.noches_disponibles, 1) AS ocupacion_pct,
       ROUND(i.ingreso / i.noches_vendidas) AS adr,
       ROUND(i.ingreso / v.noches_disponibles) AS revpar,

       ROUND(AVG(i.ingreso / v.noches_disponibles) OVER ()) AS revpar_departamental,
       CASE WHEN i.ingreso / v.noches_disponibles
              > AVG(i.ingreso / v.noches_disponibles) OVER ()
              THEN 'Por encima' ELSE 'Por debajo' END AS vs_departamento,
       ROUND(100 * i.ingreso_alta / i.ingreso, 1) AS pct_ingreso_temporada_alta,
       RANK() OVER (ORDER BY i.ingreso / v.noches_disponibles DESC) AS puesto_revpar
FROM ingreso_mun i
JOIN inventario_mun v ON v.id_municipio = i.id_municipio
JOIN municipio m ON m.id_municipio = i.id_municipio
ORDER BY puesto_revpar;

```

MUNICIPIO	NOCHES_VENDIDAS	NOCHES_DISPONIBLES	OCUPACION_PCT	ADR	REVPAR	REVPAR_DEPARTAMENTAL	VS_DEPARTAMENTO	PCT_INGRESO_TEMPORADA_ALTA	PUUESTO_REVPAR
1 Calarcá	2021	10950	18,5	427205	78848	55562	Por encima		
2 Pijao	354	2190	16,2	417291	67453	55562	Por encima		
3 Génova	764	4380	17,4	364224	63531	55562	Por encima		
4 Quimbaya	2133	12410	17,2	342019	58785	55562	Por encima		
5 Salento	2323	12410	18,7	301171	56376	55562	Por encima		
6 Montenegro	1581	9490	16,7	329548	54902	55562	Por debajo		
7 Córdoba	817	5110	16	341428	54588	55562	Por debajo		

Figura 11. Consulta 8: indicadores ADR, RevPAR y estacionalidad por municipio.

9. Validación de la entrega

Se construyó un script de validación que verifica en once bloques que la base cumple lo exigido. Los resultados obtenidos en la ejecución final, sobre una base reconstruida desde cero, son los siguientes.

9.1 Integridad referencial

Prueba	Huérfanos
Habitaciones sin alojamiento	0
Alojamientos sin municipio	0
Reservas sin cliente	0
Reservas sin habitaciones	0
Habitación de otro alojamiento	0
Pagos sin reserva	0
Tarifas sin temporada	0

9.2 Reglas de fechas y temporadas

Prueba	Filas en violación
Checkout anterior o igual al checkin	0
Reserva posterior al checkin	0
Estadías fuera del rango 2024–2026	0
Días sin temporada asignada	0
Temporadas con rango inválido	0
Temporadas superpuestas	0
Servicios fuera de la estadía	0

La prueba de temporadas superpuestas es una auto-uniión que busca cualquier par de temporadas distintas cuyos rangos se crucen. Junto con las 1.096 filas exactas de DIM_TIEMPO, demuestra que las temporadas son disjuntas y cubren el periodo sin huecos.

9.3 Solapamiento de habitaciones

Ninguna habitación aparece ocupada por dos reservas activas la misma noche: 0 pares solapados. Es la precondition del disparador de fila exigido en la Entrega 2, que no encontrará violaciones preexistentes al activarse.

9.4 Coherencia de valores

Prueba	Filas en violación
Reservas con valor cero	0
Cabecera distinta de la suma de sus habitaciones	0
Capacidad excedida	0

Prueba	Filas en violación
Pagos mayores al total de la reserva	0

9.5 Objetos creados

Tipo de objeto	Cantidad
TABLE	18 (17 tablas más la tabla de la vista materializada)
INDEX	43
SEQUENCE	15
VIEW	2
MATERIALIZED VIEW	1
JOB	1 (refresco programado de la MV)
Objetos inválidos	0

9.6 Restricciones declaradas

Tipo	Cantidad
PRIMARY KEY	17
FOREIGN KEY	17
UNIQUE	16
CHECK y NOT NULL	141

9.7 Ubicación física

12 tablas en TS_TURISMO_DAT y 6 en TS_TURISMO_HIST, conforme al diseño de la sección 6. Todos los índices residen en TS_TURISMO_IDX.

1. VOLÚMENES MÍNIMOS EXIGIDOS

TABLA	EXIGIDO	REAL ESTAD
ALOJAMIENTO	60	60 OK
CLIENTE	3000	3000 OK
DIM_TIEMPO	1096	1096 OK
HABITACION	400	400 OK
MUNICIPIO	12	12 OK
PAGO	1000	30232 OK
RESENA	500	8957 OK
RESERVA	25000	25000 OK
RESERVA_HABITACION	25000	28539 OK
RESERVA_SERVICIO	40000	40000 OK
TARIFA	1000	12960 OK

2b. COBERTURA: los 12 municipios con oferta

Ninguno puede quedar en 0, o desaparecería de las consultas de analisis.

MUNICIPIO	ALOJAMIENTOS ESTAD
Filandia	12 OK
Armenia	11 OK
Circasia	6 OK
Salento	6 OK
Calarcá	5 OK
Quimbaya	5 OK
Buenavista	4 OK
Montenegro	4 OK
Córdoba	2 OK
Génova	2 OK
La Tebaida	2 OK

4. REGLAS DE FECHAS Y TEMPORADAS (todo debe dar 0)

PRUEBA	FILAS
Checkout <= checkin	0
Reserva posterior al checkin	0
Estadias fuera de 2024-2026	0
Dias sin temporada asignada	0
Temporadas con rango invalido	0
Temporadas superpuestas	0
Servicios fuera de la estadia	0

##### 10. RESTRICCIONES DECLARADAS #####	
TIPO	CANTIDAD
CHECK / NOT NULL	141
FOREIGN KEY	17
PRIMARY KEY	17
UNIQUE	16

Figura 12. Salida completa del script de validación.

9.8 Verificación del cálculo multi-temporada

El script incluye una prueba específica que extrae una estadía concreta y muestra su valoración noche por noche, evidenciando el cambio de tarifa al cruzar de temporada:

Fecha	Temporada	Tipo	Valor noche
14/04/2024	MEDIA 2024-04-12	MEDIA	162.000
15/04/2024	MEDIA 2024-04-12	MEDIA	162.000
16/04/2024	BAJA 2024-04-16	BAJA	129.000
17/04/2024	BAJA 2024-04-16	BAJA	129.000

Las dos primeras noches se cobran a tarifa de temporada media y las dos siguientes a tarifa baja, dentro de una misma reserva. Esta es la evidencia directa de la decisión de diseño B.

##### 7. VERIFICACION DEL CALCULO MULTI-TEMPORADA #####			
## Cuantas estadias atraviesan mas de una temporada y como se cobraron.			
TEMPORADAS_ATRAVESADAS	RESERVAS	VALOR_PROMEDIO	
	1	23602	894883
	2	4883	1130642
	3	54	1540407
## Ejemplo concreto: una estadia que cruza temporadas, noche por noche.			
ID_RESERVA	FECHA	TEMPORADA	TIPO VALOR_NOCHE
6	14/04/24	MEDIA 2024-04-12	MEDIA 162000
6	15/04/24	MEDIA 2024-04-12	MEDIA 162000
6	16/04/24	BAJA 2024-04-16	BAJA 129000
6	17/04/24	BAJA 2024-04-16	BAJA 129000

Figura 13. Verificación del cálculo multi-temporada noche por noche.

10. Estructura del repositorio y orden de ejecución

El documento advierte que un script que no corre de arriba abajo en una base limpia no se califica. La secuencia fue verificada ejecutándola completa sobre una base reconstruida desde cero.

#	Script	Se ejecuta como	Qué hace
1	01_tablespaces_usuario.sql	SYS AS SYSDBA en XEPDB1	Crea los tres tablespaces, el usuario TURISMO_UQ, sus cuotas y privilegios
2	02_limpieza.sql	TURISMO_UQ	Borra objetos previos; permite re-ejecutar la secuencia completa
3	03_secuencias.sql	TURISMO_UQ	Crea las 15 secuencias
4	04_ddl.sql	TURISMO_UQ	Crea las 17 tablas con todas sus restricciones y los índices iniciales
5	05_carga_catalogos.sql	TURISMO_UQ	Municipios reales, tipos, categorías, servicios, temporadas y calendario
6	06_carga_masiva.sql	TURISMO_UQ	Alojamientos, habitaciones, tarifas, clientes, reservas, pagos, servicios y reseñas
7	07_vistas_y_mv.sql	TURISMO_UQ	Vistas de análisis y vista materializada
8	08_consultas_analisis.sql	TURISMO_UQ	Las ocho consultas obligatorias
9	09_validacion.sql	TURISMO_UQ	Checklist de verificación en once bloques

Cada script depende únicamente de objetos creados por los anteriores. El script 02 permite repetir la secuencia sin necesidad de recrear el usuario ni los tablespaces.

10.1 Índices creados y deliberadamente omitidos

Se crearon solo los índices estructuralmente necesarios: las claves foráneas de tablas hijas con borrado en cascada y alto volumen, y los caminos que usa el generador de datos.

Se omitieron a propósito, por ser el material de demostración de la Entrega 3:

- reserva(fecha_checkin, fecha_checkout), candidato a índice compuesto.
- reserva(id_alojamiento) y reserva(id_cliente), claves foráneas sin indexar.
- Cualquier índice basado en función sobre las fechas, por ejemplo TRUNC(fecha_checkin, 'MM').
- Cualquier índice sobre reserva.estado, que servirá como el caso donde el índice no ayuda: al ser COMPLETADA cerca del 80 % de las filas, el acceso por ROWID resulta más costoso que el barrido completo.

Esta omisión es una decisión consciente: si todo estuviera indexado desde la Entrega 1, no habría forma de demostrar el impacto de un índice comparando el plan de ejecución antes y después.

11. Conclusión

La Entrega 1 queda completa y verificada: el modelo cubre las 14 entidades exigidas más tres justificadas, el DDL declara 17 claves primarias, 17 foráneas, 16 restricciones UNIQUE y 141 CHECK y

NOT NULL, la carga supera todos los volúmenes mínimos, el histórico reside en su propio tablespace, y las ocho consultas obligatorias más la vista materializada producen resultados consistentes entre sí.

Las dos decisiones estructurales del proyecto quedaron resueltas de forma que las Entregas 2 y 3 se construyan sobre el mismo modelo sin reconstruirlo: la tabla puente RESERVA_HABITACION soporta reservas de varias habitaciones, y la combinación de TEMPORADA con DIM_TIEMPO permite valorar una estadia noche por noche con la tarifa de cada temporada que atraviesa, que es exactamente lo que necesitará fn_valor_estadia.